

EasyFood — Nivelamento do Projeto

Antes da próxima aula, todos devem deixar o projeto no mesmo estado desenvolvido em sala.

Depois disso, vocês terão uma nova missão relacionada à evolução da arquitetura da EasyFood.

Parte 1 — Preparando o ambiente

1. Verifique se o Node.js está instalado

Abra o terminal.

No Windows, você pode utilizar o **Prompt de Comando**, **PowerShell** ou o próprio terminal do VS Code.

No macOS ou Linux, utilize o **Terminal**.

Execute:

```
node -v
```

Depois:

```
npm -v
```

Se os dois comandos retornarem uma versão, por exemplo:

```
v22.18.0
```

e:

```
10.9.3
```

o ambiente está preparado.

Você pode seguir para a **Parte 2 — Criando o projeto EasyFood**.

2. Se o Node.js não estiver instalado

Se aparecer uma mensagem semelhante a:

```
'node' não é reconhecido como um comando
```

ou:

```
command not found: node
```

será necessário instalar o Node.js.

Acesse o site oficial:

<https://nodejs.org>

Escolha a versão **LTS — Long Term Support**.

Para nossa disciplina, utilizaremos preferencialmente a versão LTS.

Windows

Baixe o instalador e siga as etapas mantendo as configurações padrão.

Depois da instalação:

1. Feche o terminal.
2. Abra novamente.
3. Execute:

```
node -v
```

```
npm -v
```

macOS

Baixe a versão LTS correspondente ao macOS e siga o instalador.

Depois, abra novamente o terminal e execute:

```
node -v
```

```
npm -v
```

Linux

A forma de instalação pode variar conforme a distribuição.

Consulte as instruções da versão LTS disponíveis no site oficial do Node.js.

Depois da instalação:

```
node -v
```

```
npm -v
```

3. Verifique o Visual Studio Code

Também utilizaremos o **Visual Studio Code**.

Caso ainda não esteja instalado, acesse:

<https://code.visualstudio.com>

Baixe e instale a versão correspondente ao seu sistema operacional.

No VS Code, você pode abrir o terminal em:

Terminal → New Terminal

Parte 2 — Criando o projeto EasyFood

4. Crie a pasta do projeto

No terminal:

```
mkdir easy-food
```

Entre na pasta:

```
cd easy-food
```

Se quiser abrir essa pasta diretamente no VS Code e o comando estiver disponível:

```
code .
```

5. Inicie um projeto Node.js

Execute:

```
npm init -y
```

Esse comando criará o arquivo:

```
package.json
```

O `package.json` contém informações sobre o projeto, scripts e dependências utilizadas pela aplicação.

6. Instale o Express

Execute:

```
npm install express
```

O **Express** será utilizado para facilitar a criação da API da EasyFood.

Após a instalação, você perceberá que foram criados:

```
node_modules/  
package-lock.json
```

A estrutura ficará semelhante a:

```
easy-food/  
|  
├─ node_modules/  
├─ package.json  
└─ package-lock.json
```

7. Crie o arquivo `server.js`

Dentro da pasta `easy-food`, crie:

```
server.js
```

Agora teremos:

```
easy-food/  
|  
├─ node_modules/  
├─ package.json  
├─ package-lock.json  
└─ server.js
```

8. Adicione o código inicial

No arquivo `server.js`:

```
const express = require("express");  
  
const app = express();  
  
app.use(express.json());  
  
const restaurants = [  
  {  
    id: 1,  
    name: "Pizzeria Napoli",  
    category: "Pizza",  
    rating: 4.8  
  },  
  {  
    id: 2,  
    name: "Burger House",  
    category: "Hambúrguer",  
    rating: 4.7  
  },  
  {  
    id: 3,  
    name: "Sushi House",  
    category: "Japonês",  
    rating: 4.9  
  }  
];  
  
app.get("/restaurants", (req, res) => {  
  res.json(restaurants);  
});  
  
app.listen(3000, () => {
```

```
console.log("EasyFood rodando na porta 3000");
});
```

9. Execute o projeto

No terminal:

```
node server.js
```

Você deverá visualizar:

```
EasyFood rodando na porta 3000
```

Enquanto o servidor estiver executando, deixe esse terminal aberto.

10. Teste a aplicação

Abra o navegador e acesse:

```
http://localhost:3000/restaurants
```

A resposta deverá ser semelhante a:

```
[
  {
    "id": 1,
    "name": "Pizzaria Napoli",
    "category": "Pizza",
    "rating": 4.8
  },
  {
    "id": 2,
    "name": "Burger House",
    "category": "Hambúrguer",
    "rating": 4.7
  },
  {
    "id": 3,
    "name": "Sushi House",
    "category": "Japonês",
    "rating": 4.9
  }
]
```

```
}  
]
```

Se chegou até aqui:

Seu projeto está no mesmo ponto da EasyFood desenvolvida em aula.

Parte 3 — Novo requisito

O Product Owner enviou uma nova necessidade:

Os restaurantes também precisam conseguir se cadastrar na EasyFood.

Atualmente nossa aplicação permite apenas consultar restaurantes através de:

```
GET /restaurants
```

Agora precisamos pensar em como permitir a criação de um novo restaurante.

Sua missão

Antes da próxima aula, analise o sistema atual e pense nas seguintes perguntas:

- ☐ O que precisaria mudar na aplicação para permitir o cadastro de restaurantes?
 - ☐ Precisaremos criar uma nova rota?
 - ☐ Que tipo de requisição poderia ser utilizada para cadastrar um restaurante?
 - ☐ Quais informações um restaurante precisaria enviar?
 - ☐ Como essas informações chegariam até nossa API?
 - ☐ Onde o novo restaurante seria armazenado?
 - ☐ O desenho da arquitetura que criamos precisa mudar?
 - ☐ Algum componente novo seria necessário?
 - ☐ Algum conector novo seria necessário?
 - ☐ Alguma nova decisão arquitetural precisa ser tomada?
-

Pense também como arquiteto

Escolha **uma decisão** que você acredita que precisará ser tomada para implementar o novo requisito.

Responda:

- ☐ Qual problema estamos tentando resolver?
- ☐ Qual decisão você tomaria?

- [] Por que você tomaria essa decisão?
 - [] Existe outra alternativa?
 - [] Qual vantagem sua escolha traz?
 - [] Qual desvantagem ou trade-off ela pode gerar?
-

Desafio opcional

Você pode tentar implementar o cadastro de restaurantes.

Mas isso **não é obrigatório**.

O mais importante neste momento é conseguir explicar:

Como você faria e por quê?

Não se preocupe em encontrar a “resposta perfeita”.

Durante a disciplina vamos perceber que decisões arquiteturais dependem do contexto, das necessidades do sistema e dos trade-offs envolvidos.

Para a próxima aula

Chegue com:

- o projeto EasyFood funcionando;
- suas respostas para as perguntas acima;
- pelo menos uma decisão arquitetural pensada e justificada;
- e, se quiser, uma tentativa de implementação do novo requisito.

Na próxima aula vamos comparar diferentes soluções e entender como equipes de engenharia **registram e justificam decisões arquiteturais ao longo da evolução de um sistema**.